

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Архитектура вычислительных систем»

ОТЧЕТ
к лабораторной работе №1
на тему:
**«ИССЛЕДОВАНИЕ РАБОТЫ ПРОЦЕССОРА В РЕЖИМЕ ОДНОГО
ЯДРА БЕЗ ФУНКЦИИ МНОГОПОТОЧНОСТИ»**
БГУИР 6-05-0612-02 95

Выполнил студент группы 453504
РОМАНОВ Николай Викторович

(дата, подпись студента)

Проверил ассистент каф. информатики
КАЛИНОВСКАЯ Анастасия Александровна

(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

1 Осуществить методом математического выполнения функции согласно варианту задания. Значение аргумента x изменяется от a до b с шагом h . Для каждого x найти значения функции $Y(x)$, суммы $S(x)$ и число итераций n , при котором достигается требуемая точность $\varepsilon = |Y(x)-S(x)|$. Результат вывести в виде таблицы. Значения a , b , h и ε вводятся с клавиатуры.

$$3. \quad S(x) = \sum_{k=0}^n \frac{\cos(\frac{k\pi}{4})}{k!} x^k, \quad Y(x) = e^{\frac{x \cos \frac{\pi}{4}}{4}} \cos(x \sin \frac{\pi}{4})$$

Рисунок 1.1 – Заданные функции

2 Осуществить написание программы на любом удобном языке программирования с вставками кода ассемблера для выполнения задачи на i количестве итерации ($i > 5000$) для получения достоверных результатов эксперимента.

3 Результатом будут графики нагрузки на одно ядро процессора.

По оси X – время выполнения, по оси Y – количество итераций (i).

Intel – два графика на одно ядро, с функцией Hyper-Threading и без.

AMD – с функцией SMT и без.

ARM – одно низко производительное ядро и одно высокопроизводительное ядро.

2 ВЫПОЛНЕНИЕ РАБОТЫ

Для выполнения задания был написан код на языке C++, который вычисляет значение функции $Y(x)$ и $S(x)$. В программе используется вставка inline-ассемблера для математического сопроцессора (FPU) для процессоров архитектуры x86 (AMD) для оптимизации вычисления факториала, умножения и суммы. FPU специально оптимизирован для работы с числами с плавающей запятой, поэтому операции на нем выполняются более эффективно.

```
#include <iostream>
#include <fstream>
#include <cmath>
#include <iomanip>
#include <chrono>

using namespace std;

inline double asm_add(double a, double b) {
    double result;
    __asm__ __volatile__(
        "fldl %1 \n\t"
        "fldl %2 \n\t"
        "faddp %%st, %%st(1) \n\t"
        "fstpl %0 \n\t"
        : "=m" (result)
        : "m" (b), "m" (a)
    );
    return result;
}

inline double asm_mul(double a, double b) {
    double result;
    __asm__ __volatile__(
        "fldl %1 \n\t"
        "fldl %2 \n\t"
        "fmulp %%st, %%st(1) \n\t"
        "fstpl %0 \n\t"
        : "=m" (result)
        : "m" (b), "m" (a)
    );
    return result;
}

inline double asm_factorial(int n) {
    if(n == 0 || n == 1) return 1.0;
    double result = 1.0;

    for(int i = 2; i <= n; ++i) {
        double d_i = (double)i;
        __asm__ __volatile__(
            "fldl %1 \n\t"
```

```

        "fldl %0 \n\t"
        "fmlpl %%st, %%st(1) \n\t"
        "fstpl %0 \n\t"
        : "+m" (result)
        : "m" (d_i)
    );
}
return result;
}

int main() {

    double a, b, h, eps;
    long long max_iterations = 10000;

    cout << "Input a (0.1): ";
    cin >> a;
    cout << "Input b (1.0): ";
    cin >> b;
    cout << "Input h (0.1): ";
    cin >> h;
    cout << "Input eps (0.0001): ";
    cin >> eps;

    ofstream csv("data.csv");
    csv << "Time,Iterations\n";

    cout << "Starting CPU test (SMT ON/OFF)...";

    auto start_time = chrono::high_resolution_clock::now();

    for(long long i = 1; i <= max_iterations; ++i) {

        bool is_last_iteration = i == max_iterations;

        if(is_last_iteration) {
            cout << string(65, '-') << endl;
            cout << "|" << setw(8) << "x" << " |" << setw(15) << "Y(x)"
                << " |" << setw(15) << "S(x)" << " |" << setw(15) << "Итераций"
            (n) << " |" << endl;
            cout << string(65, '-') << endl;
        }

        for(double x = a; x <= b; x += h) {

            double pi_4 = M_PI / 4;
            double y_val = exp(x * cos(pi_4)) * cos(x * sin(pi_4));
            double s_val = 0.0;
            double current_term = 0.0;
            int k = 0;

            do{

                double numerator = asm_mul(cos(k * pi_4), pow(x, k));

```

```

        double denominator = asm_factorial(k);
        current_term = numerator / denominator;
        s_val = asm_add(s_val, current_term);
        ++k;

    } while(abs(y_val - s_val) >= eps && k < 100);

    if(is_last_iteration) {
        cout << "|" << fixed << setprecision(2) << setw(8) << x
            << " |" << fixed << setprecision(8) << setw(15) << y_val
            << " |" << fixed << setprecision(8) << setw(15) << s_val
            << " |" << setw(15) << (k - 1) << " |" << endl;
    }
}

if (is_last_iteration) {
    cout << string(65, '-') << endl;
}

if(i % 100 == 0) {
    auto current_time = chrono::high_resolution_clock::now();
    chrono::duration<double> elapsed = current_time - start_time;
    csv << elapsed.count() << "," << i << endl;
}

}

csv.close();

cout << "Succesfull create data.csv" << endl;

return 0;
}

```

Данная лабораторная была выполнена на ОС Kali Linux, поэтому все команды будут относиться именно к этому дистрибутиву.

Перед запуском программы ее нужно было скомпилировать, поэтому я использовал команду `g++ -O0 main.cpp -o lab1`, где `-O0` указывает не оптимизировать исходный код, т.к. это может сказаться на результате.

Данный код был запущен в 2 режимах: с функцией SMT и без нее.

Первоначальный запуск программы в эмуляторе терминала (при переключении в ТТУ через `Ctrl+Alt+F1`) показал искаженные результаты из-за влияния фоновых процессов графической оболочки. Для получения достоверных метрик производительности и полного исключения влияния графического сервера, параметры загрузки ядра Linux (GRUB) были модифицированы: добавлен параметр 3, что обеспечило загрузку системы в строгом текстовом режиме (`multi-user.target`).

По умолчанию функция SMT уже включена, поэтому сразу запускаем программу, используя команду `taskset -c 0 ./lab1`. По результатам выполнения построим график зависимости количества выполненных итераций от времени выполнения программы.

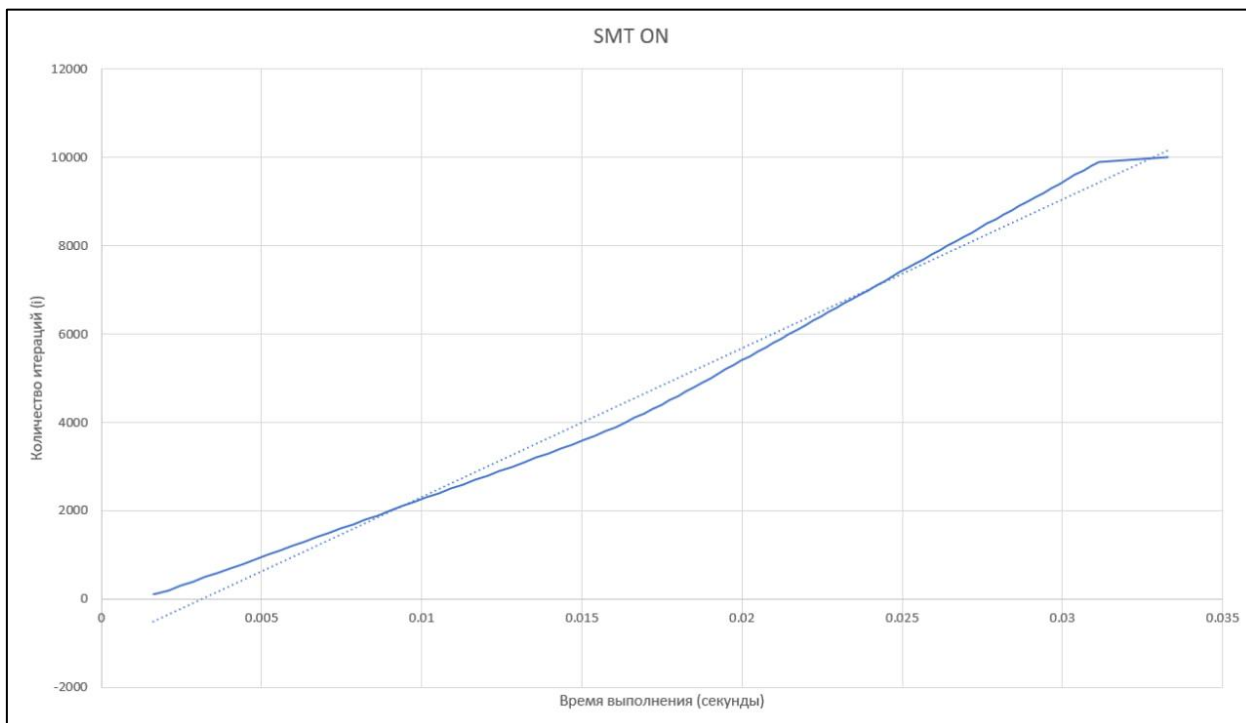


Рисунок 2.1 – График выполнения в одноядерном режиме с функцией SMT

Для запуска программы без функции SMT нужно ее принудительно отключить, сделать это можно с помощью команды `sudo tee /sys/devices/system/cpu/smt/control`, позднее с помощью этой же команды можно включить функцию SMT назад. Запускаем программу используя все ту же команду `taskset -c 0 ./lab1`.

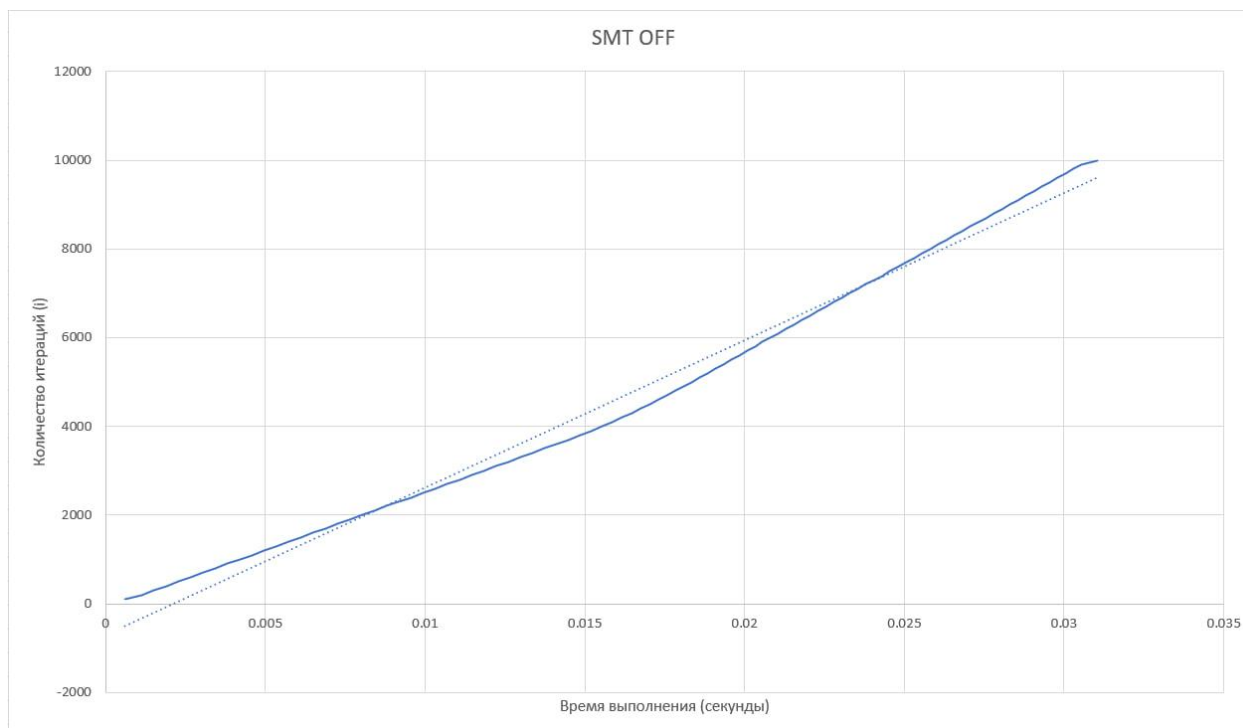


Рисунок 2.2 – График выполнения в одноядерном режиме без функции SMT

ВЫВОД

В ходе эксперимента были получены графики зависимости времени выполнения математической задачи (с интенсивным использованием FPU) от количества итераций на одном ядре процессора AMD.

Результаты показали, что в изолированной среде (без GUI) выполнение программы с отключенной функцией SMT происходит быстрее (0.0310 с. против 0.0333 с. на конечных итерациях). Особенно ярко преимущество монопольного режима ядра проявилось на начальном этапе выполнения задачи, где время выполнения может отличаться в 3 раза.

Данный результат полностью согласуется с теорией архитектуры вычислительных систем. Технология SMT (Simultaneous Multithreading) подразумевает разделение ресурсов одного физического ядра (таких как L1-кэш, блок предсказания ветвлений, очереди инструкций и конвейеры FPU) между двумя логическими потоками.

При включенном SMT аппаратное обеспечение вынуждено поддерживать структуры для двух потоков, что вносит небольшие накладные расходы (overhead). При ручном отключении SMT математический алгоритм получает абсолютную монополию на все исполнительные блоки и кэш-память физического ядра. Отсутствие необходимости аппаратного разделения ресурсов и конкуренции за кэш привело к закономерному снижению времени выполнения однопоточной задачи.